PUBLIC

Code Assessment

of the cUSDO Solana Smart Contracts

January 10th, 2025

Produced for

OpenEden

by

CHAINSECURITY

Contents

1 Executive Summary 3
2 Assessment Overview 5
3 Limitations and use of report 8
4 Terminology 9
5 Findings 10
6 Resolved Findings 11

1 Executive Summary

Dear OpenEden Team,

Thank you for trusting us to help OpenEden with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of cUSDO Solana according to Scope to support you in forming an opinion on their security risks.

OpenEden implements a transfer hook for cUSDO a wrapped non-rebasing version of USDO. The transfer hook allows the admins of the token to pause and unpause token transfers.

The most critical subjects covered in our audit are access control and functional correctness. Security regarding all the aforementioned subjects is high. Only minor issues were uncovered. All the issues have been addressed.

The general subjects covered are gas efficiency, specification and testing. Security regarding all the aforementioned subjects is high.

In summary, we find that the codebase provides a high level of security.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,

ChainSecurity

1.1 Overview of the Findings

Below we provide a brief numerical overview of the findings and how they have been addressed.

Critical-Severity Findings	0
High-Severity Findings	0
Medium-Severity Findings	0
Low-Severity Findings	1
• Code Corrected	1

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The assessment was performed on the source code files inside the cUSDO Solana repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

V	Date	Commit Hash	Note
1	Oct 30th, 2024	d24e0448f65d50d69606582014f7d25ebd627e0f	Initial Version
2	Dec 3rd, 2024	72923548617e8fb8ad899a1fad12b28ed1ea3115	Fixes
3	Dec 9th, 2024	00336be06ddc85364ba5bc28149a517b1fb23d45	Extra Fixes
4	Dec 20th, 2024	2ca5430ae51858e8296f057398361fda8721f3ea	Final Version

For the Solana programs, the Anchor framework `0.30.1` was used with Rust `1.81.0`

The following program was in scope:

`programs/cusdo/lib.rs`

2.1.1 Excluded from scope

All programs not explicitly in scope are considered out of scope. We assume that the Anchor framework works according to specification. The Rust compiler is assumed to correctly expand the various macros and compile the Solana program. Attack vectors that involve trusted users such as the admin and the operators were not considered. The implementation of the transfer-hook was compared to the transfer-hook implementation in Anchor's documentation which is assumed to be correct. The underlying economical model of the system is out of scope.

2.2 System Overview

This system overview describes the initially received version (Version 1) of the contracts as defined in the Assessment Overview.

Furthermore, in the findings section, we have added a version icon to each of the findings to increase the readability of the report.

OpenEden offers a transfer-hook extension for token-2022. The extension makes the token pausable. Moreover, the Solana program implements a bonus multiplier logic which scales the amount of tokens a user holds. The multiplier can be increased by whitelisted operators. The program is meant to be used to implement the logic required for cUSDO. cUSDO is a wrapped non-rebasing version of USDO.

2.2.1 Functionality

The program exposes the following functionality:

`initialize_extra_account_meta_list()`: called by the token-2022 via a CPI and initializes the transfer hook. Every program which needs to initialize particular logic for the hook needs to implement this function. This function can only be called in a transaction where the signer is the admin of the contract.

`update_pause()`: the admin of the contract can pause or unpause the transfers. The state of the token is represented by an account that stores whether the token is paused or not. The account is initialized by this call, therefore `initialize_extra_account_meta_list` can only be called successfully after calling `update_pause()`.

`transfer_hook()`: as the name suggests it's called when the token is transferred. It fails if the token is paused or the pause account hasn't been initialized yet.

`fallback()`: the standard implementation of the fallback function that eventually calls the transfer hook.

`update_bonus_multiplier()`: the admin can initialize or reset the bonus multiplier to an arbitrary value. The bonus multiplier is stored in a separate account.

`add_bonus_multiplier()`: a whitelisted operator can increase the bonus multiplier by a specific amount.

`add_role()`: the admin adds a public key to the whitelisted operators. A whitelisted operator is represented by an account containing its public address.

`delete_role()`: the admin deletes an account that represents an operator essentially removing a public key from the whitelisted operators.

`get_usdo_amount()`: for any account of any mint (token), it returns the number of tokens of a user multiplied by the bonus multiplier.

2.2.2 Trust Model and Assumptions

We infer the following trust model for the system:

- Admin: this user is in full control of the system, they can pause and unpause the token, they can add and remove operators which are users with increased privileges and they can reset the bonus multiplier. Furthermore, the admin is the upgrade authority of the program allowing it to modify it at any time. Therefore, they are fully trusted.
- Operators: they can increase the bonus multiplier by an arbitrary amount. They are assumed to be fully trusted.
- Everyone: arbitrary signers can only call `get_usdo_amount` and indirectly invoke the hook. There are no trust assumptions for them.
- We assume that the underlying financial mechanism supporting USDO is correct and we do not consider cases where USDO loses its peg or gets devalued. Furthermore, we assume that the rate does not decrease in such a way that it causes issues in the system (e.g. falling below BASE in the beginning)

2.2.3 Changes in Version 2

The following changes were made in Version 2:

- The state of the token, now referred to as USDO account, is initialized in a separate transaction by the admin (`initialize_usdo`). The account stores the `pause` field and the cUSDO address in the USDO field. The state is extended to store the address of the USDO mint making the program reusable in case of an upgrade. Additionally, this stored address is used to ensure that only the USDO mint account can be passed as the mint account to function calls such as `get_usdo_amount()` or `transfer_hook()` . The address can be

updated by the admin by calling `update_usdo_address`. It's important to note that during the initialization of the account, the fields are not set. To set them, it is required to call the respective update functions. We assume that the update is part of the deployment process. `initialize_usdo` can be called multiple times successfully setting the `bump` to the same value and emitting the same event.

- The initialization of the state `BonusMultiplier` happens with a `initialize_bonus_multiplier`. As with `initialize_usdo`, the fields of the account are not set to their initial values. We expected the admin to call `update_bonus_multiplier` to set the correct initial value of the multiplier. `initialize_bonus_multiplier` can be called multiple times setting the bump to the same value and emitting the same event.

3 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.

4 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- *Likelihood* represents the likelihood of a finding to be triggered or exploited in practice
- *Impact* specifies the technical and business-related consequences of a finding
- *Severity* is derived based on the likelihood and the impact

We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

5 Findings

In this section, we describe any open findings. Findings that have been resolved have been moved to the Resolved Findings section. The findings are split into these different categories:

- Correctness: Mismatches between specification and implementation

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical-Severity Findings	0
High-Severity Findings	0
Medium-Severity Findings	0
Low-Severity Findings	0

6 Resolved Findings

Here, we list findings that have been resolved during the course of the engagement. Their categories are explained in the Findings section.

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical-Severity Findings	0
High-Severity Findings	0
Medium-Severity Findings	0
Low-Severity Findings	1

- get_usdo_amount() Accepts Arbitrary Mints Code Corrected

Informational Findings	3

- Optimizations Code Corrected
- State Change Is Not Logged Code Corrected
- Transfer Hook Can Be Invoked by Anyone Code Corrected

6.1 `get_usdo_amount()` Accepts Arbitrary Mints

Correctness Low Version 1 Code Corrected

CS-SOLUSDO-004

`get_usdo_amount()` is used to calculate the actual value of cUSDO in USDO. The method accepts an asscociated token account (ATA) and a mint. However, it is never checked that the ATA belongs to this particular mint. Moreover, there's no check that the mint corresponds to cUSDO. Therefore, a user can pass an arbitrary mint account and an arbitrary ATA. Note that in the context of this program, there's no security risk as the `get_usdo_amount` is a view method.

Code corrected:

The function can only be called with the appropriate token accounts

6.2 Optimizations

Informational Version 1 Code Corrected

CS-SOLUSDO-002

1. **Redundant check in ``add_bonus_multiplier``**

`add_bonus_multiplier()` should check that the signer of the transaction is an operator. The check is performed in the following lines:

```
#[derive(Accounts)]
pub struct AddBonusMultiplier<'info> {
#[account(mut)]
```

```
pub payer: Signer<'info>,

...

#[account(
    seeds = [&payer.key().to_bytes()],
    bump,
)]
pub role: Account<'info, Role>,
}
```

The snippet above implements the following logic: it checks that the public address of the signer of the transaction is used as a seed for a Role account. Note that if such an account doesn't exist the transaction will fail. In other words, this check is enough to guarantee that the signer is indeed an operator. Therefore, the following check of `add_bonus_multiplier` is redundant as it's trivially true when we reach this point:

```
require!(
 ctx.accounts.role.operator == ctx.accounts.payer.key(),
 Error::NotOperator
);
```

2. **``bump`` not stored**

For every PDA computed from the canonical bump in the program the bump value is not stored in the PDA account but recomputed every time the PDA is used. While this is not a security risk it is considered best practice to store the bump value in the PDA account.

3. **Redundant account passing**

There are multiple instances where redundant accounts are passed to the methods and never used:

- `InitializeExtraAccountMetaList` accepts a `token_program`
- `GetUSDOAmount` accepts a `mint`

Code corrected:

1. The redundant check was removed as well as the `operator` field of the `role` account.
2. Bumps are now stored for all accounts.
3. Redundancy has been eliminated.

6.3 State Change Is Not Logged

Informational **Version 1** **Code Corrected**

CS-SOLUSDO-001

`update_pause()` and `update_bonus_multiplier()` do not log any message when the state is changed. However, `add_role()` and `transfer_hook()` log an informational message when the state is changed.

No public logs will be available when the pause status or bonus multiplier is updated. This may make it difficult to track changes to these states.

Code corrected:

The appropriate events are now emitted.

6.4 Transfer Hook Can Be Invoked by Anyone

Informational | Version 1 | Code Corrected

CS-SOLUSDO-003

`transfer_hook()` does not implement any additional access control checks. Therefore, any mint can use the implemented hook or anyone can call the hook directly. While the hook functionality is limited to preventing transfers if the `pause` flag is set this does not pose a direct security risk. However, a message is emitted every time the hook is called successfully which will reflect all successful transfers from all mints using this hook and not just the USDO mint. This could be misleading to entities relying on the emitted messages to track transfers.

Code corrected:

The required checks have been added to make sure that only the correct accounts should be used by the hook. Moreover, the hook sanitzes the `amount` argument to be greater than 0. The transfer hook makes use of `assert_is_transferring` which returns an error if the transfer hook isn't called during a transfer.